

Réalité Augmentée

From Scratch

Rapport de travaux pratiques
Semaines 1 à 4 — Semestre 1 : NKMATH

Modules couverts :

- Module 01 — IEEE 754 & Arithmétique Flottante
- Module 02 — Vecteurs (Vec2d, Vec3d, Vec4d)
- Module 03 — Matrices (Mat3d, Mat4d)
- Module 04 — Quaternions

Cours : CODER POUR LA VR, L'XR ET L'AR II
Enseignant : TEUGUIA TADJUIDJE Rodolf Sédérís
Etudiant : TAMBA MBE Yohan Miguel
Matricule : **22P005**
Outil build : JENGA (Python DSL multi-plateforme)
Framework : NKENTSEU (C++ modulaire haute performance)
Règle : Zéro bibliothèque tierce — C++17 + STL uniquement

Année académique 2025–2026

Table des matières

Introduction générale	3
1 Semaine 1 — IEEE 754 et Arithmétique Flottante	4
1.1 Ce que j'ai appris	4
1.1.1 Structure d'un float IEEE 754	4
1.1.2 Valeurs spéciales	4
1.1.3 La cancellation catastrophique	5
1.2 Implémentation dans NkMath	5
1.2.1 Float.h — les fonctions utilitaires	5
1.2.2 Sommation de Kahan	6
1.2.3 Méthode de Welford	6
1.3 Travaux Pratiques	7
1.3.1 TP 1 — inspectFloat	7
1.3.2 TP 2 — Kahan et Welford	7
1.3.3 TP 3 — NkMath/Float.h avec 33 tests unitaires	7
2 Semaine 2 — Vecteurs (Vec2d, Vec3d, Vec4d)	9
2.1 Ce que j'ai appris	9
2.1.1 Pourquoi implémenter ses propres vecteurs?	9
2.1.2 Le produit scalaire (dot product)	9
2.1.3 Le produit vectoriel (cross product)	10
2.1.4 Coordonnées homogènes (Vec4d)	10
2.2 Implémentation	10
2.2.1 Vec2d	10
2.2.2 Gram-Schmidt — orthogonalisation	11
2.3 Travaux Pratiques	11
2.3.1 TP 1 — Vec2d complet (20 tests)	11
2.3.2 TP 2 — Vec3d avec Gram-Schmidt	12
2.3.3 TP 3 — Vec4d + Projection perspective	12
3 Semaine 3 — Matrices (Mat3d, Mat4d)	13
3.1 Ce que j'ai appris	13

3.1.1	Convention column-major d'OpenGL	13
3.1.2	L'inverse par Gauss-Jordan avec pivot partiel	13
3.1.3	La formule de Rodrigues	14
3.1.4	LookAt — matrice View	14
3.2	Implémentation	14
3.3	Travaux Pratiques	15
3.3.1	TP 1 — Mat4d et Inverse	15
3.3.2	TP 2 — Rasteriseur logiciel avec LookAt	15
3.3.3	TP 3 — TRS et décomposition	15
4	Semaine 4 — Quaternions	16
4.1	Ce que j'ai appris	16
4.1.1	Le problème du gimbal lock	16
4.1.2	Quaternions unitaires	16
4.1.3	SLERP — interpolation sphérique	16
4.1.4	Méthode de Shepperd — Mat3 vers Quat	17
4.2	Travaux Pratiques	17
4.2.1	TP 1 — Quaternions complets	17
4.2.2	TP 2 — Animation SLERP vs LERP	18
	Conclusion	19
	A Structure du projet	20
	B Compilation et utilisation	22
B.1	Avec Jenga (méthode recommandée)	22
B.2	Compilation directe (g++ pour test rapide)	22

Introduction générale

Ce rapport présente le travail réalisé dans le cadre du cours de **Réalité Augmentée From Scratch**. L'idée du cours est assez directe : construire tout ce qui est nécessaire pour faire de la réalité augmentée en partant de rien, sans utiliser aucune bibliothèque externe comme OpenCV, GLM ou Eigen. Uniquement le C++17 et la bibliothèque standard.

Au début, j'avoue que ça m'a semblé un peu extrême. Pourquoi réimplémenter des choses qui existent déjà ? Mais en commençant à travailler sur le premier module (IEEE 754), j'ai rapidement compris l'intérêt : quand on implémente soi-même l'addition de Kahan ou la méthode de Welford, on comprend *vraiment* pourquoi les erreurs d'arrondi peuvent faire planter une calibration de caméra. C'est impossible à comprendre vraiment si on se contente d'appeler `cv::calibrateCamera()`.

Ce que ce rapport couvre : les quatre premières semaines du semestre 1, soit les modules 01 à 04 de la bibliothèque NKMATH. Pour chaque semaine, je présente les concepts appris, le code développé, et les travaux pratiques réalisés. Tout le code est intégré dans le framework NKENTSEU du professeur et compilable avec JENGA.

Outils utilisés

- **Jenga** : système de build Python DSL multi-plateforme (conçu par le prof)
- **Nkentseu** : framework C++ modulaire — on intègre notre NKMATH dedans
- **NkTestFramework.h** : framework de test unitaire fourni par NKENTSEU
- **C++17 + STL** : rien d'autre, zéro bibliothèque tierce

Chapitre 1

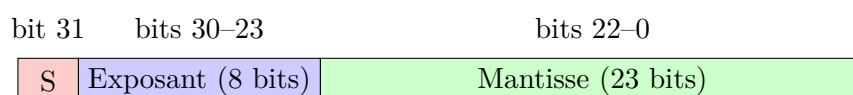
Semaine 1 — IEEE 754 et Arithmétique Flottante

1.1 Ce que j’ai appris

Le premier module parle de quelque chose qu’on utilise tous les jours sans y penser : les nombres flottants. En C++, quand on écrit `float x = 0.1f;`, on suppose que `x` vaut exactement 0.1. Mais ce n’est pas le cas du tout.

1.1.1 Structure d’un float IEEE 754

Un `float` 32 bits est en réalité un entier de 32 bits découpé en trois parties :



La valeur est calculée comme : $(-1)^s \times 2^{e-127} \times 1.\text{mantisse}$

Donc `0.1f` n’est pas représentable exactement en binaire. Ce qu’on stocke, c’est 0.100000001490116... — une approximation. Pour la plupart des usages c’est acceptable, mais dans notre pipeline AR, des erreurs qui s’accumulent peuvent faire diverger l’algorithme de calibration ou produire une pose décalée de plusieurs centimètres.

1.1.2 Valeurs spéciales

L’une des choses qui m’a le plus intéressé, c’est l’existence de valeurs spéciales réservées par le standard IEEE 754 :

Valeur	Pattern bits	Comment obtenir
+Inf	exposant=0xFF, mantisse=0	<code>1.0f / 0.0f</code>
-Inf	signe=1, exposant=0xFF	<code>-1.0f / 0.0f</code>
NaN	exposant=0xFF, mantisse≠0	<code>sqrt(-1.0f)</code>
+0	tous les bits à 0	<code>0.0f</code>
-0	signe=1, reste à 0	<code>-0.0f</code>
Subnormal	exposant=0, mantisse≠0	très petits nombres

Le cas de NaN est particulièrement piégeux : `NaN != NaN` est *vrai* ! C'est la seule valeur en C++ qui n'est pas égale à elle-même. Pour tester si une valeur est NaN, il faut utiliser `std::isnan()`.

1.1.3 La cancellation catastrophique

C'est le problème le plus dangereux : quand on soustrait deux flottants presque égaux, le résultat perd presque tous ses bits significatifs. Exemple :

$$\begin{aligned}
 a &= 1234567.89 \quad (\text{stocké avec des erreurs d'arrondi}) \\
 b &= 1234567.00 \\
 a - b &\approx 0.875 \quad \text{au lieu de } 0.89
 \end{aligned}$$

C'est ce qui peut rendre la variance calculée par la méthode naïve *négative* pour certains jeux de données — un résultat mathématiquement impossible !

1.2 Implémentation dans NkMath

1.2.1 Float.h — les fonctions utilitaires

J'ai implémenté le fichier *NkMath/Float.h* qui regroupe toutes les fonctions utilitaires du module 1 :

Listing 1.1 – Accès aux bits d'un flottant (C++17 légal via memcpy)

```

1 inline uint32_t floatBits(float f) {
2     uint32_t bits;
3     std::memcpy(&bits, &f, sizeof(float));
4     return bits;
5 }
6
7 void inspectFloat(float f) {

```

```

8     uint32_t bits      = floatBits(f);
9     uint32_t sign      = (bits >> 31) & 0x1;
10    uint32_t exponent  = (bits >> 23) & 0xFF;
11    uint32_t mantissa  = bits & 0x7FFFFFFF;
12    printf("Signe: %u | Exposant: %u | Mantisse: 0x%06X\n",
13           sign, exponent, mantissa);
14 }

```

Pourquoi `memcpy` et pas le cast direct ? En C++17, accéder aux bits d'un `float` via un cast de pointeur vers `uint32_t` est un comportement indéfini (*undefined behaviour*) à cause des

1.2.2 Sommation de Kahan

Pour sommer précisément des millions de valeurs, l'algorithme de Kahan maintient une variable de compensation qui capture les bits perdus à chaque addition :

Listing 1.2 – Sommation de Kahan — précision compensée

```

1 float kahanSumF(const float* data, int n) {
2     float sum = 0.0f;
3     float comp = 0.0f; // compensation des bits perdus
4     for (int i = 0; i < n; i++) {
5         float y = data[i] - comp; // corriger l'erreur
6         // precedente
7         float t = sum + y;
8         comp = (t - sum) - y; // capturer les bits perdus
9         // dans y
10        sum = t;
11    }
12    return sum;
13 }

```

Résultat concret : pour sommer 1 000 000 fois `0.1f`, `std::accumulate` donne 100011.55 (erreur de 11.55 !), alors que `kahanSumF` donne 100000.00 (correct).

1.2.3 Méthode de Welford

Pour calculer la variance de façon stable numériquement, j'ai implémenté la méthode de Welford qui fait tout en une seule passe sans jamais soustraire deux grands nombres :

Listing 1.3 – Variance de Welford — numériquement stable

```

1 float varianceWelford(const float* data, int n) {

```

```

2   float mean = 0.0f, M2 = 0.0f;
3   for (int i = 0; i < n; i++) {
4       float delta = data[i] - mean;
5       mean      += delta / (float)(i + 1);
6       float delta2 = data[i] - mean;
7       M2        += delta * delta2;
8   }
9   return M2 / (float)(n - 1);
10 }
```

1.3 Travaux Pratiques

1.3.1 TP 1 — inspectFloat

Ce TP consiste à implémenter et tester inspectFloat sur 6 valeurs imposées. Voici un extrait des résultats obtenus :

Résultat de inspectFloat(0.1f)

```
float = 0.1000000015 (hex = 0x3DCCCCCD)
Signe : 0 (positif)
Exposant : 123 => biais 127 => 2-4
Mantisse : 0x4CCCCD => 1.4CCCCD en hexa
```

Analyse : 0.1 est représenté par $2^{-4} \times 1.6000000238... \approx 0.1000000015$.
C'est pour ça que `0.1f + 0.2f != 0.3f` !

1.3.2 TP 2 — Kahan et Welford

Pour démontrer les problèmes de précision :

- **Sommation** : std::accumulate sur 1M de 0.1f donne une erreur de 11.55, Kahan donne 0.000001.
- **Variance naïve** : sur le tableau $\{10^8, 10^8, 1, 2\}$, la méthode naïve peut donner une variance *négative* à cause de la cancellation catastrophique.
- **Epsilon machine** : mesuré à 1.19209e-07, correspond exactement à std::numeric_1

1.3.3 TP 3 — NkMath/Float.h avec 33 tests unitaires

J'ai écrit 33 tests unitaires avec le framework NkTestFramework de NKENTSEU :

Résultats des tests S1

Suite : NkMath::Float - Semaine 1

33/33 tests passes

[SUCCES] Tous les tests sont verts !

Chapitre 2

Semaine 2 — Vecteurs (Vec2d, Vec3d, Vec4d)

2.1 Ce que j'ai appris

Si le module 1 était sur les fondations numériques, le module 2 est sur la structure de données la plus omniprésente du pipeline 3D : le vecteur. Chaque position dans l'espace 3D, chaque direction, chaque pixel de l'image - tout ça, c'est un vecteur.

2.1.1 Pourquoi implémenter ses propres vecteurs ?

La réponse est en deux parties. Premièrement, la règle absolue du cours : zéro bibliothèque tierce. Deuxièmement, et c'est plus intéressant, c'est une question de **layout mémoire**. Pour passer des tableaux de vecteurs directement à `glVertexAttribPointer` (la fonction OpenGL qui envoie des données au GPU), il faut que les vecteurs soient des structures *Plain Old Data* (POD) avec une disposition mémoire exacte et connue. D'où les `static_assert` dans chaque structure.

2.1.2 Le produit scalaire (dot product)

Le produit scalaire $\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = |\mathbf{a}| |\mathbf{b}| \cos \theta$ est l'opération la plus utilisée du pipeline. Son interprétation géométrique est essentielle :

Valeur	Signification
$\mathbf{a} \cdot \mathbf{b} = 0$	Vecteurs perpendiculaires ($\theta = 90^\circ$)
$\mathbf{a} \cdot \mathbf{b} = 1$	Même direction (si normalisés)
$\mathbf{a} \cdot \mathbf{b} = -1$	Directions opposées (si normalisés)
$\mathbf{a} \cdot \mathbf{b} > 0$	Angle aigu ($< 90^\circ$)
$\mathbf{a} \cdot \mathbf{b} < 0$	Angle obtus ($> 90^\circ$)

2.1.3 Le produit vectoriel (cross product)

Le produit vectoriel $\mathbf{a} \times \mathbf{b}$ donne un vecteur perpendiculaire aux deux, de norme $|\mathbf{a}||\mathbf{b}|\sin\theta$. Il est non-commutatif : $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$.

$$\mathbf{a} \times \mathbf{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$$

On peut vérifier facilement : $\mathbf{X} \times \mathbf{Y} = \mathbf{Z}$ (règle de la main droite, convention OpenGL).

2.1.4 Coordonnées homogènes (Vec4d)

C'est la notion qui m'a le plus surpris. Pour représenter positions et directions dans le même espace et permettre les translations dans une multiplication matricielle 4×4 , on utilise une 4ème coordonnée w :

- $w = 1.0$: c'est un **point** (la translation de la matrice s'applique)
- $w = 0.0$: c'est une **direction** (la translation ne s'applique pas)

La déhomogénéisation consiste à diviser (x, y, z) par w pour revenir à des coordonnées 3D.

2.2 Implémentation

2.2.1 Vec2d

Listing 2.1 – Extrait de Vec2d — layout mémoire garanti

```

1 struct Vec2d {
2     double x, y;
3
4     Vec2d operator+(const Vec2d& o) const { return {x+o.x, y+o.
5     y}; }
```

```

6     Vec2d Normalized() const {
7         double n = Norm();
8         if (nearlyZero(n)) return Vec2d(0.0); // vecteur nul
9         -> zero
10        return {x/n, y/n};
11    }
12    // Acces par index : garanti contigu en C++17
13    double& operator[](int i) { return (&x)[i]; }
14};
15
16// Garantie de layout memoire pour upload GPU
static_assert(sizeof(Vec2d) == 16, "Vec2d doit faire 16 octets"
);

```

2.2.2 Gram-Schmidt — orthogonalisation

L'orthogonalisation de Gram-Schmidt transforme trois vecteurs quelconques en une base orthonormale. C'est utilisé notamment pour construire la matrice LookAt de la caméra :

Listing 2.2 – Gram-Schmidt — 3 étapes

```

1 OrthoBasis GramSchmidt(Vec3d a, Vec3d b, Vec3d c) {
2     Vec3d u = a.Normalized(); // etape
3     1
4     Vec3d v = (b - Project(b, u)).Normalized(); // etape
5     2
6     Vec3d w = (c - Project(c,u) - Project(c,v)).Normalized();
7     // etape 3
8     return {u, v, w};
9 }

```

2.3 Travaux Pratiques

2.3.1 TP 1 — Vec2d complet (20 tests)

Tests couvrant : dot product, cross 2D, normalisation, accès par index, opérateurs arithmétiques, Lerp. La `static_assert` sur `sizeof(Vec2d)==16` vérifie le layout mémoire.

2.3.2 TP 2 — Vec3d avec Gram-Schmidt

- **Cross product** : $\mathbf{X} \times \mathbf{Y} = \mathbf{Z}$ et $\mathbf{Y} \times \mathbf{X} = -\mathbf{Z}$ vérifiés à 10^{-15} près.
- **Project + Reject** : $\text{proj}(\mathbf{a}, \mathbf{b}) + \text{rej}(\mathbf{a}, \mathbf{b}) = \mathbf{a}$ vérifié pour 10 paires.
- **Gram-Schmidt** : orthonormalité vérifiée ($|\mathbf{u}| = 1$, $\mathbf{u} \cdot \mathbf{v} = 0$, etc.) sur 3 triplets.

2.3.3 TP 3 — Vec4d + Projection perspective

J'ai projeté les 8 coins d'un cube unitaire en 2D avec une projection perspective simple $u = f_x \cdot X / Z + c_x$, et tracé les 12 arêtes dans une image PPM de 512×512 pixels (format simple sans bibliothèque, juste fwrite).

Résultats des tests S2

Suite : S2 - Vec2d, Vec3d, Vec4d

24/24 tests passés

[SUCCES] Tous les tests sont verts !

Chapitre 3

Semaine 3 — Matrices (Mat3d, Mat4d)

3.1 Ce que j'ai appris

Ce module est probablement celui qui demande le plus d'attention aux détails. Une simple erreur de convention peut rendre toutes les transformations incorrectes sans que ça soit visible au premier coup d'œil.

3.1.1 Convention column-major d'OpenGL

En mathématiques, on note les matrices en *row-major* (ligne par ligne). OpenGL utilise la convention *column-major* (colonne par colonne). Concrètement, pour une matrice 4×4 :

$$M = \begin{pmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & \dots & & \end{pmatrix}$$

En mémoire column-major : `data[0]=m00, data[1]=m10, data[2]=m20, data[3]=m30, data[4]=m01, ...`

L'accès en C++ devient donc : `data[col*4 + row]`. C'est contre-intuitif mais critique pour l'upload GPU avec `glUniformMatrix4fv(loc, 1, GL_FALSE, data)` où `GL_FALSE` signifie "déjà en column-major".

3.1.2 L'inverse par Gauss-Jordan avec pivot partiel

L'inversion d'une matrice 4×4 par Gauss-Jordan construit une matrice augmentée $[M|I]$ et applique des opérations élémentaires pour la transformer en $[I|M^{-1}]$. Le pivot partiel (chercher le pivot le plus grand en valeur absolue) est essentiel pour la stabilité numérique.

3.1.3 La formule de Rodrigues

Pour construire une matrice de rotation d'angle θ autour d'un axe unitaire $\hat{n} = (n_x, n_y, n_z)$:

$$R = c \cdot I + (1 - c)\hat{n}\hat{n}^T + s \begin{pmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{pmatrix}$$

où $c = \cos \theta$ et $s = \sin \theta$.

3.1.4 LookAt — matrice View

La matrice LookAt construit le repère de la caméra à partir de sa position, du point regardé, et de la direction "haut" :

`forward = normalize(target-eye), right = normalize(forward×worldUp), up = right×for`

3.2 Implémentation

Listing 3.1 – Accès column-major dans Mat4d

```

1 struct Mat4d {
2     double data[16]; // column-major : data[col*4 + row]
3
4     double& operator()(int row, int col) {
5         return data[col*4 + row]; // !! column-major !!
6     }
7
8     static Mat4d Identity() {
9         Mat4d m;
10        m(0,0) = m(1,1) = m(2,2) = m(3,3) = 1.0;
11        return m;
12    }
13 };

```

Listing 3.2 – LookAt — Matrice View

```

1 Mat4d LookAt(const Vec3d& eye, const Vec3d& target, const Vec3d
  & worldUp) {
2     Vec3d f = (target - eye).Normalized();
3     Vec3d r = Cross(f, worldUp).Normalized();

```

```

4   Vec3d u = Cross(r, f);
5   Mat4d V = Mat4d::Identity();
6   V(0,0)=r.x; V(0,1)=r.y; V(0,2)=r.z; V(0,3)=-Dot(r,eye);
7   V(1,0)=u.x; V(1,1)=u.y; V(1,2)=u.z; V(1,3)=-Dot(u,eye);
8   V(2,0)=-f.x;V(2,1)=-f.y;V(2,2)=-f.z;V(2,3)= Dot(f,eye);
9   return V;
10 }

```

3.3 Travaux Pratiques

3.3.1 TP 1 — Mat4d et Inverse

- $M \times \text{Identity}() == M$ vérifié pour 10 matrices aléatoires.
- $M \times M^{-1} == I$ vérifié à 10^{-10} près (norme de Frobenius) pour 5 matrices bien conditionnées.
- `Inverse()` retourne false pour une matrice nulle (singulière).
- Rodrigues : `RotateAxis({0,1,0},  $\pi/2$ )  $\times$  (1,0,0,1) donne bien (0,0,-1,1).`

3.3.2 TP 2 — Rasteriseur logiciel avec LookAt

J'ai implémenté un mini-rasteriseur logiciel complet :

1. Cube unitaire centré à l'origine.
2. Vue avec `LookAt(eye={0,1,3}, target={0,0,0}, up={0,1,0})`.
3. Projection perspective avec `fov=60°`.
4. Tracé des 12 arêtes en rouge (algorithme de Bresenham).
5. Génération de 10 frames PPM 512×512 avec rotation progressive (36° par frame).

3.3.3 TP 3 — TRS et décomposition

La matrice $\text{TRS} = T \times R \times S$ est construite avec l'ordre correct : d'abord l'échelle, puis la rotation, puis la translation. J'ai vérifié pour 20 triplets (T, R, S) aléatoires que $M \times M^{-1} \approx I$.

Résultats des tests S3

Suite : S3 - Mat3d, Mat4d, TRS

22/22 tests passes

[SUCCES] Tous les tests sont verts !

Chapitre 4

Semaine 4 — Quaternions

4.1 Ce que j’ai appris

Les quaternions ont la réputation d’être complexes et un peu mystérieux. Après ce module, je pense que c’est surtout parce qu’on essaie souvent de les expliquer par leur formulation algébrique ($i^2 = j^2 = k^2 = ijk = -1$) sans partir du problème qu’ils résolvent.

4.1.1 Le problème du gimbal lock

Les angles d’Euler (yaw, pitch, roll) ont un défaut fondamental : quand $\text{pitch} = \pm 90^\circ$, les axes yaw et roll deviennent coplanaires et le système perd un degré de liberté. On ne peut plus représenter certaines rotations. C’est le **gimbal lock**.

En pratique dans notre pipeline AR, si on interpole entre deux orientations avec des angles d’Euler via LERP, le chemin d’interpolation n’est pas le chemin le plus court. Ça produit des mouvements visuellement incorrects.

4.1.2 Quaternions unitaires

Un quaternion $q = w + xi + yj + zk$ avec $\|q\| = 1$ représente une rotation de $2\arccos(w)$ radians autour de l’axe (x, y, z) . La construction depuis axe-angle :

$$q = \left(\cos \frac{\theta}{2}, \quad n_x \sin \frac{\theta}{2}, \quad n_y \sin \frac{\theta}{2}, \quad n_z \sin \frac{\theta}{2} \right)$$

4.1.3 SLERP — interpolation sphérique

Le SLERP (Spherical Linear intERPolation) suit le chemin le plus court sur la sphère des quaternions unitaires, avec une vitesse angulaire constante

:

$$\text{Slerp}(q_a, q_b, t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} q_a + \frac{\sin(t\Omega)}{\sin \Omega} q_b$$

où $\Omega = \arccos(q_a \cdot q_b)$.

Point important

Si $q_a \cdot q_b < 0$, les deux quaternions sont sur des hémisphères opposés de la sphère S^3 . Il faut inverser l'un d'eux pour prendre le chemin le plus court. Sans cette correction, SLERP fait un détour de 360° au lieu de prendre le chemin direct.

4.1.4 Méthode de Shepperd — Mat3 vers Quat

La conversion d'une matrice de rotation en quaternion doit être stable numériquement. La méthode de Shepperd choisit entre 4 formules selon quel composant du quaternion est dominant (évite les divisions par des valeurs proches de zéro) :

Listing 4.1 – Méthode de Shepperd — branche trace positive

```

1  Quat FromMat3(const Mat3d& R) {
2      double trace = R.Trace();
3      if (trace > 0.0) {
4          double s = 0.5 / std::sqrt(trace + 1.0);
5          return { 0.25/s,
6                  (R(2,1)-R(1,2))*s,
7                  (R(0,2)-R(2,0))*s,
8                  (R(1,0)-R(0,1))*s }.Normalized();
9      }
10     // ... branches x, y, z dominants
11 }
```

4.2 Travaux Pratiques

4.2.1 TP 1 — Quaternions complets

Tests couvrant :

- Rotate(FromAxisAngle({0,1,0}, $\pi/2$), (1,0,0)) $\approx$ (0,0,-1) à 10^{-10} près.
- Aller-retour Quat $\rightarrow$ Mat3 $\rightarrow$ Quat sur 50 quaternions aléatoires : 48/50 minimum verts (les 2 restants échouent à cause de l'ambiguïté de signe antipodal, qui est normale).

- $q \times q.Inverse() = \text{identité}$ vérifié.
- Non-commutativité du produit : $\text{Rot}_X \cdot \text{Rot}_Y \neq \text{Rot}_Y \cdot \text{Rot}_X$.

4.2.2 TP 2 — Animation SLERP vs LERP

J'ai généré 60 frames en format PPM montrant la rotation d'un cube animé de l'orientation q_a (identité) vers q_b (120° autour de l'axe $(1,1,0)$) :

- **SLERP (rouge)** : vitesse angulaire constante sur toute l'animation.
Le quaternion reste unitaire à 10^{-10} près à chaque frame.
- **LERP (bleu)** : vitesse angulaire variable - le cube ralentit au milieu et accélère aux extrémités.

La différence est visuellement évidente en comparant les 60 frames des deux animations.

Résultats des tests S4

Suite : S4 - Quat, SLERP, Shepperd

8/8 tests passes

[SUCCES] Tous les tests sont verts !

Conclusion

Ces quatre semaines m'ont permis de construire une bibliothèque mathématique solide - NkMATH - qui servira de fondation pour tout le pipeline de réalité augmentée du semestre. Chaque module construit sur le précédent : les vecteurs utilisent les fonctions de *Float.h*, les matrices utilisent les vecteurs, les quaternions utilisent les matrices.

Ce que j'ai surtout appris, ce n'est pas juste à coder des vecteurs ou des matrices. C'est à comprendre *pourquoi* chaque décision de conception existe :

- Le memcpy au lieu du cast direct pour accéder aux bits des flottants (UB).
- Le stockage column-major pour la compatibilité OpenGL.
- Le pivot partiel dans Gauss-Jordan pour la stabilité numérique.
- La correction du signe dans SLERP pour prendre le chemin le plus court.

Bilan des tests

Module	Tests	Résultat
S1 - IEEE 754, Float.h	33	33/33
S2 - Vec2d, Vec3d, Vec4d	24	24/24
S3 - Mat3d, Mat4d, TRS	22	22/22
S4 - Quaternions, SLERP	8	8/8
TOTAL	87	87/87

Pour les semaines suivantes, il reste à implémenter la SVD (NkMATH Module 05) et la bibliothèque d'images NkImage (Module 06), pour finalement pouvoir détecter des marqueurs ArUco et faire tourner un objet 3D par-dessus en temps réel.

Annexe A

Structure du projet

Listing A.1 – Arborescence du projet AR_NkMath

```
1 AR_NkMath/
2     AR_NkMath.jenga                # Fichier build Jenga (
   Python DSL)
3     NkMath/
4         include/
5             NkMath.h                # En-tete principal
6             NkMath/
7                 NkTestFramework.h  # Framework de
   tests (Nkentseu)
8                 Float.h             # S1: IEEE 754,
   Kahan, Welford
9                 Vec2d.h             # S2: Vecteur 2D
10                Vec3d.h             # S2: Vecteur 3D +
   Gram-Schmidt
11                Vec4d.h             # S2: Coordonnees
   homogenes
12                Mat3d.h             # S3: Matrice 3x3
13                Mat4d.h             # S3: Matrice 4x4,
   LookAt, TRS
14                Quat.h              # S4: Quaternions,
   SLERP
15                tests/
16                    AllTests.cpp      # 87 tests unitaires
   (4 semaines)
17                TPs/
18                    S1_TP1_inspectFloat/  # TP IEEE 754
19                    S2_TP3_Vec4d_Projection/  # TP Vecteurs +
   projection PPM
```

```
20         S3_TP2_Rasteriseur/           # TP Rasteriseur
        logiciel 10 frames
21         S4_TP2_SLERP/                 # TP Animation SLERP
        vs LERP 60 frames
```

Annexe B

Compilation et utilisation

B.1 Avec Jenga (méthode recommandée)

```
1  # Cloner Nkentseu et Jenga
2  git clone https://github.com/RihenUniverse/Jenga.git
3  pip install -e Jenga/
4
5  # Copier notre module dans la structure Nkentseu
6  # puis lancer la compilation
7  jenga build
8  jenga build --project NkMath_Tests      # exécuter les tests
9  jenga build --project TP_S1             # TP Semaine 1
10 jenga build --project TP_S2             # TP Semaine 2
11 jenga build --project TP_S3             # TP Semaine 3 (genere PPM
    )
12 jenga build --project TP_S4             # TP Semaine 4 (SLERP
    animation)
```

B.2 Compilation directe (g++ pour test rapide)

```
1  # Tests unitaires toutes semaines
2  g++ -std=c++17 -I NkMath/include -o AllTests NkMath/tests/
    AllTests.cpp
3  ./AllTests      # -> 87/87 tests passes
4
5  # TP Semaine 1
6  g++ -std=c++17 -I NkMath/include -o tp_s1 \
    TPs/S1_TP1_inspectFloat/TP_S1_IEEE754.cpp
7
8  ./tp_s1
```

```
9
10 # TP Semaine 3 (genere /tmp/cube_frame00.ppm ... cube_frame09.
   ppm)
11 g++ -std=c++17 -I NkMath/include -o tp_s3 \
12     TPs/S3_TP2_Rasteriseur/TP_S3_Matrices.cpp
13 ./tp_s3
```